

Lab: Client-Server Application - Event Catalog

50 Points Possible

Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

7/14/2026

▼ Details



Lab Client-Server Application - Event Catalog

In this lab, you'll create a lightweight web app that links a front end built with HTML and JavaScript to a Flask-powered back end. Your objectives are to:

- Set up a Flask server that handles both GET and POST requests.
- Use the `fetch()` API in JavaScript to send and receive data from the server.
- Return clean, consistent JSON responses from Flask routes.
- Dynamically render and update content on the front end based on data from the back end.

This lab strengthens your understanding of how the front end and back end interact through HTTP requests and JSON-based messaging.

Tools and Resources

GitHub Repo: [Lab - Client Server Application- Event Catalog \(https://github.com/learn-co-curriculum/course-8-module-6-connect-client-server-lab\)](https://github.com/learn-co-curriculum/course-8-module-6-connect-client-server-lab)

Instructions

Set Up

- Fork and clone the GitHub repository to your machine.
- Open the project in VSCode or your preferred editor.

- Start your back-end Flask server: `python app.py`.
- Open the `index.html` file in your browser.

Task 1: Define the Problem

You're building a simple events catalog app. The user should be able to:

- See a list of events from the back end.
- Submit a new event using a form.
- See that the new event immediately appears on the page.

Task 2: Determine the Design

Your application will:

- Use Flask to serve two endpoints:
 - **GET** `/`: return a welcome message
 - **GET** `/events`: return all event data
 - **POST** `/events`: accept and store new event data
- Store event information in a Python list.
- Accept JSON on POST and respond with JSON.
- Use JavaScript to fetch event data and update the DOM.

Task 3: Develop, Test, and Refine the Code

Step 1: Create a Feature Branch

Before writing code, create a new feature branch:

```
git checkout -b feature-connect-frontend
```

Step 2: Build the Flask Server

Update `app.py` with the following logic:

```
from flask import Flask, request, jsonify

app = Flask(__name__)

events = [
    {"id": 1, "title": "Yoga in the Park"},
    {"id": 2, "title": "Lake 5K Run"}
]

@app.route("/", methods=["GET"])
def welcome():
    return jsonify({"message": "Welcome!"}), 200

@app.route("/events", methods=["GET"])
def get_events():
    return jsonify(events), 200

@app.route("/events", methods=["POST"])
def add_event():
    data = request.get_json()
    new_id = max((e["id"] for e in events), default=0) + 1
    new_event = {"id": new_id, "title": data["title"]}
    products.append(new_event)
    return jsonify(new_event), 201

if __name__ == "__main__":
    app.run(debug=True)
```

Step 3: Build the Front End Interaction

Update your `script.js` file with logic like this:

```
fetch("http://localhost:5000/events")
    .then(response => response.json())
    .then(events => {
        events.forEach(renderEvent);
    });

document.querySelector("form").addEventListener("submit", (e) => {
    e.preventDefault();
    const title = document.querySelector("#title").value;

    fetch("http://localhost:5000/events", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ title })
    })
    .then(response => response.json())
    .then(renderEvent);
});
```

```
function renderEvent(event) {  
  const li = document.createElement("li");  
  li.textContent = event.title;  
  document.querySelector("#event-list").appendChild(li);  
}
```

Step 4: Push and Merge the Feature

```
git add .  
git commit -m "Implement front end and back end connection"  
git push origin feature-connect-frontend  
git checkout main  
git pull origin main  
git merge feature-connect-frontend  
git branch -d feature-connect-frontend
```

Task 4: Document and Maintain

Best Practices

- Always return structured JSON from Flask routes.
- Use JavaScript to validate input before sending a request.
- Use `request.get_json()` on the server to access client data.
- Keep logic separated: one file for front end, one for back end.
- Use print statements or dev tools to debug as needed.

Considerations

Input Validation

Challenge: What if the user submits a form without filling out required fields?

Route or Resource Not Found

Challenge: What should happen if a user tries to access a resource that doesn't exist (e.g., wrong URL or missing data)?

Solution: On the back end, check that required keys (like title) are present in the JSON body, and return a 400 Bad Request if they're missing.

Trade-off: Adds conditional logic to each route, but prevents silent failures and improves clarity for the user.

Solution: Return a 404 Not Found response with a meaningful error message.

Trade-off: Adds an extra check for valid data, but ensures the front end gets feedback it can use.

Code Reuse

Challenge: You might need to write the same lookup or response formatting logic multiple times.

Solution: Write small helper functions to centralize common tasks like finding an object by ID.

Trade-off: Slight increase in abstraction, but leads to cleaner, easier-to-maintain code.

Scaling the Project

Challenge: As you add features, your single `app.py` file could grow cluttered and hard to navigate.

Solution: Consider separating routes, logic, and data into different modules or packages.

Trade-off: Increases setup complexity early on, but sets the stage for larger, more organized projects.

⚠ Important

Before you submit your solution, you need to save your progress with git.

1. Add your changes to the staging area by executing `git add`.
2. Create a commit by executing `git commit -m "Your commit message"`
3. Push your commits to GitHub by executing `git push origin main` or `git push origin master`, depending on the name of your branch.

Submission and Grading Criteria

1. Use the rubric below as a guide for how this lab is graded.
2. Your submission will be automatically scored in CodeGrade, using the most recent commit. Remember to make sure you have pushed your commit to GitHub before submitting your assignment. Please note, you have **two attempts** to submit, and all criteria must be met to pass the AutoTest.
3. You can review your submission in CodeGrade and see your final score in your Canvas gradebook.
4. When you are ready to submit, complete your submission in the CodeGrade window below.
 - Click on + Create Submission. Connect your repository for this lab.
 - For additional information on submitting assignments in CodeGrade: [Getting Started in Canvas \(https://help.codegrade.com/for-students/getting-started/getting-started-in-canvas\)](https://help.codegrade.com/for-students/getting-started/getting-started-in-canvas).

✓ View Rubric

Lab: Client-Sever Application- Event Catalog		
Criteria	Ratings	
Application renders a homepage with welcome message view longer description	Autotest Passed	Autotest Does Not Pass
Application handles a request and returns array view longer description	Autotest Passed	Autotest Does Not Pass
Application handles a POST request and returns a 201 array view longer description	Autotest Passed	Autotest Does Not Pass

6/19/26, 5:08 AMLab: Client-Server Application - Event Catalog

Criteria	Ratings	
Created response with a valid JSON object view longer description	Autotest Passed	Autotest Does Not Pass
JavaScript call retrieves data and updates the DOM view longer description	Autotest Passed	Autotest Does Not Pass
JavaScript fetch() call sends a request and renders the new event to the page view longer description	Autotest Passed	Autotest Does Not Pass

🏠 > 📁 Genpact_Software Engineering_API Development with Python > 📁 Lab: Client-Server Application - Event Catalog ⚡

📁 Lab: Client-Server Application - Event Catalog

No deadline • Locks in 25 days • Grades published

Your Submission

Create submission

+ Upload files

Upload files from your computer to create a new submission

📁

+ Connect Git

Connect a git repo to create submissions as you commit

📁

© 2026 - CodeGrade (Revanche) - Made with ❤️ - Privacy statement - Help

<

(https://bletchley.instructure.com/courses/2180/modules/items/293486)

>

(https://bletchley.instructure.com/courses/2180/mo

https://bletchley.instructure.com/courses/2180/assignments/37477?module_item_id=2934887/7